Name- Huzaifa Maqbool	Academic Task No.-1
Roll Number- B44	Academic Task Type: Individual
Registration Number- 12512435	Course Title: Big Data Analytics
Date of allotment: 24/04/2026	Maximum Marks: 50
Date of submission: 30/04/2026	Submitted To: Rinki Gupta Maam

Learning Outcomes: This assignment enabled me to develop a comprehensive understanding of the end-to-end big data processing pipeline within the Hadoop ecosystem. I gained practical experience in data ingestion using Sqoop, where I learned how to efficiently transfer structured data between relational databases and HDFS. Through the use of Apache Pig, I developed the ability to perform data transformation and analytical operations such as filtering, grouping, sorting, and projection on large datasets using a data flow approach. Additionally, working with Apache Hive enhanced my understanding of data warehousing concepts, including the creation of internal and external tables and the implementation of static and dynamic partitioning techniques to optimize query performance. Overall, this assignment strengthened my technical proficiency in handling large-scale data, improved my analytical thinking, and provided valuable insights into designing scalable and efficient data processing solutions in real-world scenarios.

Declaration: I declare that this Assignment is my individual work. I have not copied it from any other students' work or from any other source except where due acknowledgement is made explicitly in the text, nor has any part been written for me by any other person.

Evaluation Criterion: Rubrics on different parameters

Evaluator's Comments:

General Observations	Suggestions For Improvement	Best Part Of Assignment

TASK 1: DATA IMPORT AND EXPORT USING SQOOP

Project Definition

The objective of this task is to understand and implement data transfer operations between a relational database and Hadoop Distributed File System (HDFS) using Sqoop. This involves importing structured data from MySQL into HDFS and exporting processed data back to the relational database, thereby demonstrating bidirectional data movement in a big data environment.

Outcomes / Learning

This task enhanced my understanding of data ingestion techniques in big data ecosystems. I gained practical exposure to transferring structured datasets between MySQL and HDFS using Sqoop commands. Additionally, I developed clarity on parallel data transfer using mappers and understood how Hadoop integrates with traditional database systems for scalable data processing.

Required Tools

- **Tool Used:** Apache Sqoop, Hadoop (HDFS), MySQL
- **Technologies Involved:**
    - Sqoop (Data transfer tool)
    - HDFS (Storage layer)
    - MySQL (Relational database)

Working

Step 1: Database Setup in MySQL

A database named *lpu* was created in MySQL. Three tables were defined:

- students
- faculty
- students_export

The students table consisted of attributes such as id, name, marks, and branch. Sample records were inserted into both students and faculty tables to simulate real-world data. The students_export table was created with the same schema as the students table to store exported data.

Step 2: Import a Single Table from MySQL to HDFS

Sqoop import command was executed to transfer the students table from MySQL into HDFS. The data was stored in the directory /user/cloudera/students.

This process involved launching a MapReduce job, where data was divided and transferred into HDFS. A _SUCCESS file and data file (part-m-00000) confirmed successful import.

Step 3: Import All Tables Excluding One Table

The import-all-tables command was used to transfer all tables from the database into HDFS while excluding the faculty table. The data was stored in a warehouse directory.

This demonstrated selective data ingestion and efficient handling of multiple tables using Sqoop.

Step 4: Export Data from HDFS to MySQL

Sqoop export command was used to transfer data from HDFS back into the MySQL table students_export. The data was read from HDFS and inserted into the relational table using a specified delimiter.

The successful execution ensured that the data in students_export matched the original dataset.

Output Explanation

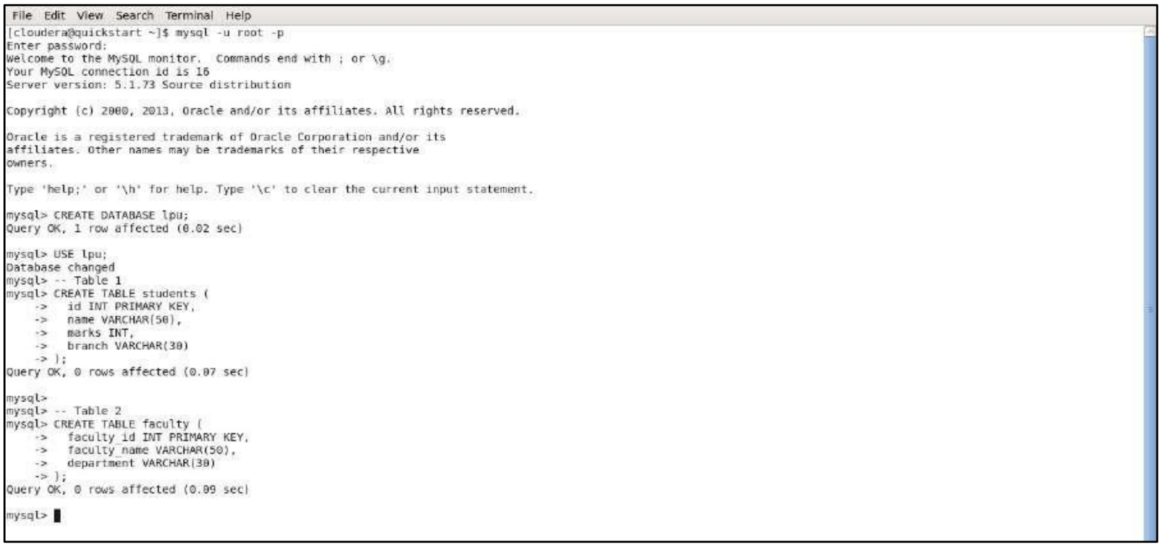
- The import operation successfully transferred all records from MySQL into HDFS, verified using HDFS commands.
- The bulk import operation excluded the specified table correctly.
- The export operation accurately transferred data back into MySQL without any data loss.
- Data consistency was maintained across all operations, confirming the correctness of the Sqoop commands.

Learning Outcome (Task 1)

I learned how to perform efficient data transfer between relational databases and Hadoop using Sqoop. This task strengthened my understanding of data ingestion, parallel processing using MapReduce, and integration of traditional database systems with distributed storage frameworks.

Database Setup in MySQL

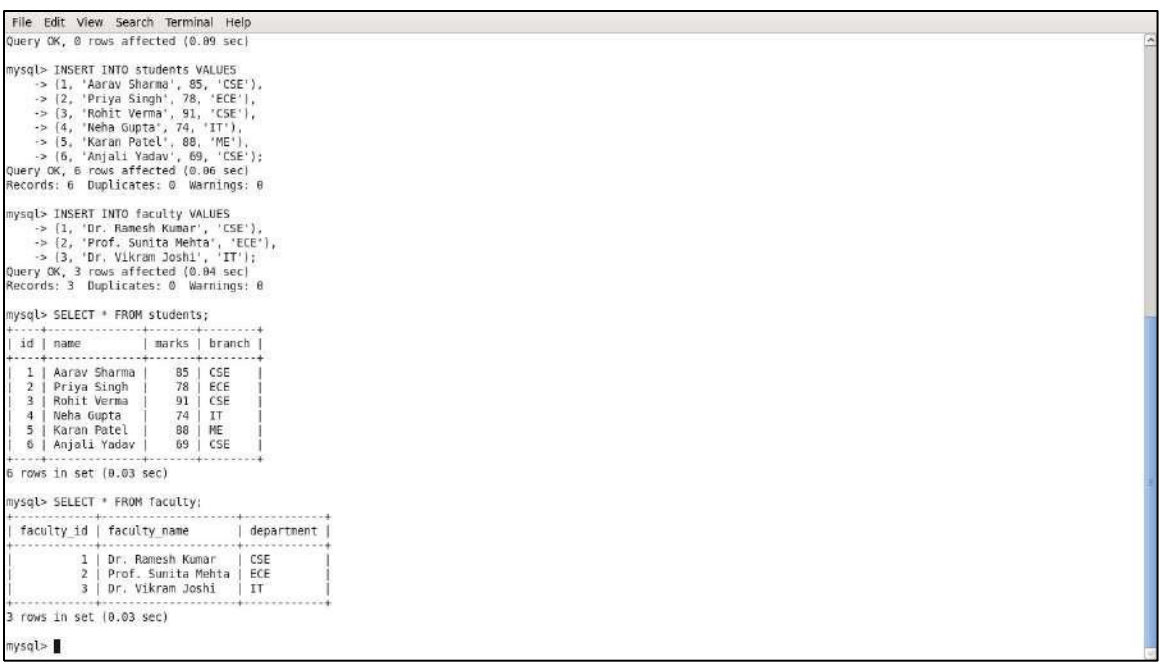
- First we logged into MySQL and created a database called lpu. Then we made three tables: students, faculty, and students_export. The students table has id, name, marks, and branch columns.



```
File  Edit  View  Search  Terminal  Help
[cloudera@quickstart ~]$ mysql -u root -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 16
Server version: 5.1.73 Source distribution

Copyright (c) 2000, 2013, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE lpu;
Query OK, 1 row affected (0.02 sec)

mysql> USE lpu;
Database changed
mysql> -- Table 1
mysql> CREATE TABLE students (
    ->   id INT PRIMARY KEY,
    ->   name VARCHAR(50),
    ->   marks INT,
    ->   branch VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.07 sec)

mysql>
mysql> -- Table 2
mysql> CREATE TABLE faculty (
    ->   faculty_id INT PRIMARY KEY,
    ->   faculty_name VARCHAR(50),
    ->   department VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.09 sec)

mysql>
```

Screenshot 1: MySQL login, database lpu created, and tables students and faculty created.

- Next we inserted 6 student records and 3 faculty records, then ran SELECT * to confirm the data is there.



```
File  Edit  View  Search  Terminal  Help
Query OK, 0 rows affected (0.09 sec)

mysql> INSERT INTO students VALUES
    -> (1, 'Aarav Sharma', 85, 'CSE'),
    -> (2, 'Priya Singh', 78, 'ECE'),
    -> (3, 'Rohit Verma', 91, 'CSE'),
    -> (4, 'Neha Gupta', 74, 'IT'),
    -> (5, 'Karan Patel', 88, 'ME'),
    -> (6, 'Anjali Yadav', 69, 'CSE');
Query OK, 6 rows affected (0.06 sec)
Records: 6  Duplicates: 0  Warnings: 0

mysql> INSERT INTO faculty VALUES
    -> (1, 'Dr. Ramesh Kumar', 'CSE'),
    -> (2, 'Prof. Sunita Mehta', 'ECE'),
    -> (3, 'Dr. Vikram Joshi', 'IT');
Query OK, 3 rows affected (0.04 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM students;
+----+--------------+-------+--------+
| id | name         | marks | branch |
+----+--------------+-------+--------+
|  1 | Aarav Sharma |    85 | CSE    |
|  2 | Priya Singh  |    78 | ECE    |
|  3 | Rohit Verma  |    91 | CSE    |
|  4 | Neha Gupta   |    74 | IT     |
|  5 | Karan Patel  |    88 | ME     |
|  6 | Anjali Yadav |    69 | CSE    |
+----+--------------+-------+--------+
6 rows in set (0.03 sec)

mysql> SELECT * FROM faculty;
+------------+--------------------+------------+
| faculty_id | faculty_name       | department |
+------------+--------------------+------------+
|          1 | Dr. Ramesh Kumar   | CSE        |
|          2 | Prof. Sunita Mehta | ECE        |
|          3 | Dr. Vikram Joshi   | IT         |
+------------+--------------------+------------+
3 rows in set (0.03 sec)

mysql>
```

Screenshot 2: Data inserted into students and faculty tables. SELECT confirms 6 students and 3 faculty rows.

- We also created the students_export table (same structure as students) and ran SHOW TABLES to confirm all 3 tables exist in lpu.

```
File  Edit  View  Search  Terminal  Help

mysql> SELECT * FROM students;
+----+--------------+-------+--------+
| id | name         | marks | branch |
+----+--------------+-------+--------+
|  1 | Aarav Sharma |    85 | CSE    |
|  2 | Priya Singh  |    78 | ECE    |
|  3 | Rohit Verma  |    91 | CSE    |
|  4 | Neha Gupta   |    74 | IT     |
|  5 | Karan Patel  |    88 | ME     |
|  6 | Anjali Yadav |    69 | CSE    |
+----+--------------+-------+--------+
6 rows in set (0.03 sec)

mysql> SELECT * FROM faculty;
+------------+--------------------+------------+
| faculty_id | faculty_name       | department |
+------------+--------------------+------------+
|          1 | Dr. Ramesh Kumar   | CSE        |
|          2 | Prof. Sunita Mehta | ECE        |
|          3 | Dr. Vikram Joshi   | IT         |
+------------+--------------------+------------+
3 rows in set (0.03 sec)

mysql> CREATE TABLE students_export (
    ->   id INT PRIMARY KEY,
    ->   name VARCHAR(50),
    ->   marks INT,
    ->   branch VARCHAR(30)
    -> );
Query OK, 0 rows affected (0.10 sec)

mysql> SHOW TABLES;
+-----------------+
| Tables_in_lpu   |
+-----------------+
| faculty         |
| students        |
| students_export |
+-----------------+
3 rows in set (0.04 sec)

mysql> 
```

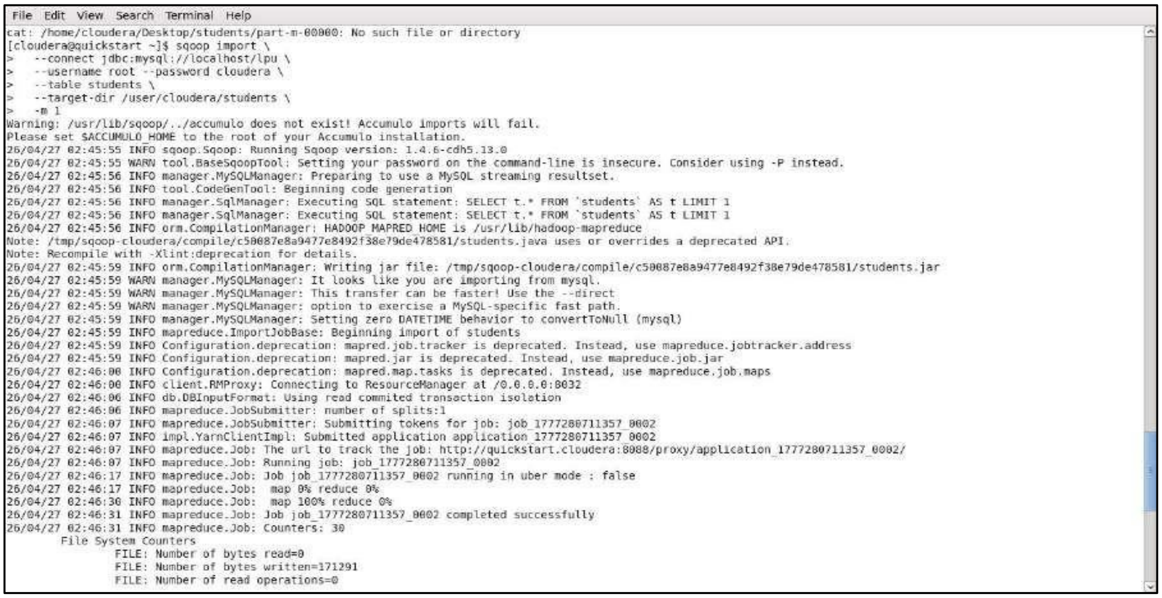
Screenshot 3: students_export table created. SHOW TABLES shows all 3 tables: faculty, students, students_export.

Q1: Import a Single Table from MySQL to HDFS

Command used:

sqoop import with --connect, --username, --password, --table students, --target-dir/user/cloudera/students, -m 1

This pulls the students table from MySQL into HDFS at the path /user/cloudera/students. The -m 1 flag means we use only 1 mapper.



Screenshot 4: Sqoop import command running. MapReduce job starts — map 0% then 100%.

- After the job finishes, we checked HDFS with hdfs dfs -ls. We can see a _SUCCESS file and part-m-00000 (the actual data file, 125 bytes).

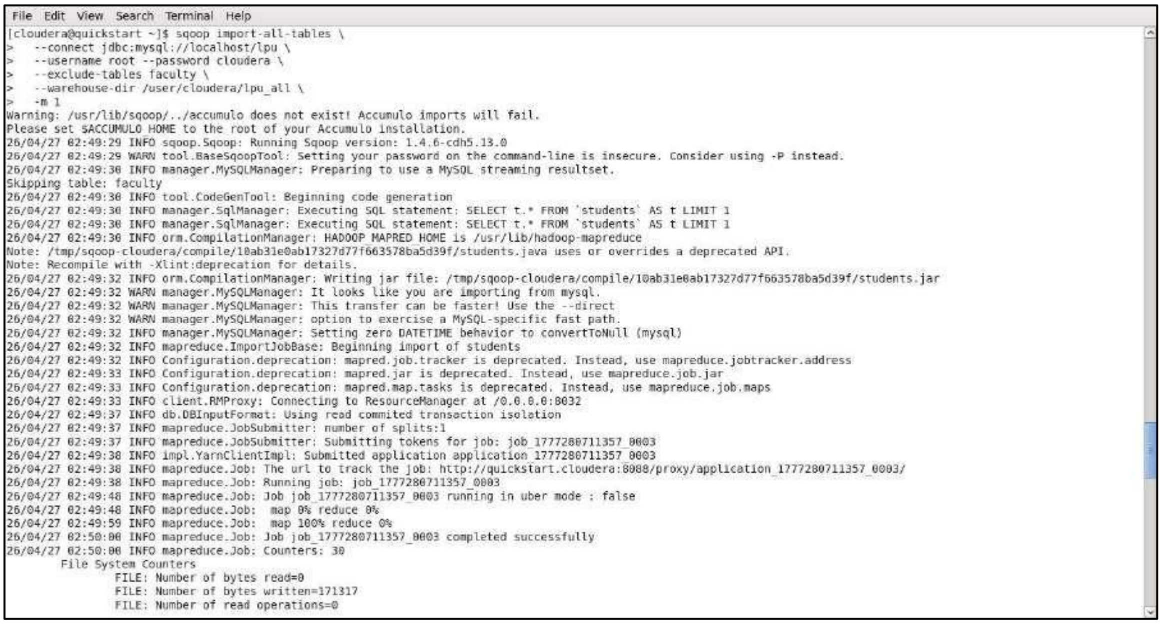
Screenshot 5: Import complete — 6 records transferred. HDFS shows _SUCCESS and part-m-00000 at/user/cloudera/students.

Q2: Import All Tables Excluding Faculty

Command used:

sqoop import-all-tables with --exclude-tables faculty and --warehouse-dir /user/cloudera/lpu_all

This imports every table in the lpu database except faculty. The output goes into /user/cloudera/lpu_all. You can see in the logs: "Skipping table: faculty".



```
File  Edit  View  Search  Terminal  Help
[cloudera@quickstart ~]$ sqoop import-all-tables \
>   --connect jdbc:mysql://localhost/lpu \
>   --username root --password cloudera \
>   --exclude-tables faculty \
>   --warehouse-dir /user/cloudera/lpu_all \
>   -m 1
Warning: /usr/lib/sqoop/../accumulo does not exist! Accumulo imports will fail.
Please set $ACCUMULO_HOME to the root of your Accumulo installation.
26/04/27 02:49:29 INFO sqoop.Sqoop: Running Sqoop version: 1.4.6-cdh5.13.0
26/04/27 02:49:29 WARN tool.BaseSqoopTool: Setting your password on the command-line is insecure. Consider using -P instead.
26/04/27 02:49:30 INFO manager.MySQLManager: Preparing to use a MySQL streaming resultset.
Skipping table: faculty
26/04/27 02:49:30 INFO tool.CodeGenTool: Beginning code generation
26/04/27 02:49:30 INFO manager.SqlManager: Executing SQL statement: SELECT t.* FROM `students` AS t LIMIT 1
26/04/27 02:49:30 INFO manager.SqlManager: Executing SQL statement: SELECT t.* FROM `students` AS t LIMIT 1
26/04/27 02:49:30 INFO orm.CompilationManager: HADOOP_MAPRED_HOME is /usr/lib/hadoop-mapreduce
Note: /tmp/sqoop-cloudera/compile/10ab31e0ab17327d77f663578ba5d39f/students.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.
26/04/27 02:49:32 INFO orm.CompilationManager: Writing jar file: /tmp/sqoop-cloudera/compile/10ab31e0ab17327d77f663578ba5d39f/students.jar
26/04/27 02:49:32 WARN manager.MySQLManager: It looks like you are importing from mysql.
26/04/27 02:49:32 WARN manager.MySQLManager: This transfer can be faster! Use the --direct
26/04/27 02:49:32 WARN manager.MySQLManager: option to exercise a MySQL-specific fast path.
26/04/27 02:49:32 INFO manager.MySQLManager: Setting zero DATETIME behavior to convertToNull (mysql)
26/04/27 02:49:32 INFO mapreduce.ImportJobBase: Beginning import of students
26/04/27 02:49:32 INFO Configuration.deprecation: mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address
26/04/27 02:49:33 INFO Configuration.deprecation: mapred.jar is deprecated. Instead, use mapreduce.job.jar
26/04/27 02:49:33 INFO Configuration.deprecation: mapred.map.tasks is deprecated. Instead, use mapreduce.job.maps
26/04/27 02:49:33 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032
26/04/27 02:49:37 INFO db.DBInputFormat: Using read commited transaction isolation
26/04/27 02:49:37 INFO mapreduce.JobSubmitter: number of splits:1
26/04/27 02:49:37 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1777280711357_0003
26/04/27 02:49:38 INFO impl.YarnClientImpl: Submitted application application_1777280711357_0003
26/04/27 02:49:38 INFO mapreduce.Job: The url to track the job: http://quickstart.cloudera:8088/proxy/application_1777280711357_0003/
26/04/27 02:49:38 INFO mapreduce.Job: Running job: job_1777280711357_0003
26/04/27 02:49:48 INFO mapreduce.Job: Job job_1777280711357_0003 running in uber mode : false
26/04/27 02:49:48 INFO mapreduce.Job:  map 0% reduce 0%
26/04/27 02:49:59 INFO mapreduce.Job:  map 100% reduce 0%
26/04/27 02:50:00 INFO mapreduce.Job: Job job_1777280711357_0003 completed successfully
26/04/27 02:50:00 INFO mapreduce.Job: Counters: 30
        File System Counters
                FILE: Number of bytes read=0
                FILE: Number of bytes written=171317
                FILE: Number of read operations=0
```

Screenshot 6: import-all-tables running. Sqoop skips faculty and imports students. MapReduce job submits.

- Job finished. We ran hdfs dfs -cat to read the imported file and confirmed all 6 student records are there.

```
File  Edit  View  Search  Terminal  Help
                FILE: Number of large read operations=0
                FILE: Number of write operations=0
                HDFS: Number of bytes read=87
                HDFS: Number of bytes written=0
                HDFS: Number of read operations=4
                HDFS: Number of large read operations=0
                HDFS: Number of write operations=2
        Job Counters
                Launched map tasks=1
                Other local map tasks=1
                Total time spent by all maps in occupied slots (ms)=11443
                Total time spent by all reduces in occupied slots (ms)=0
                Total time spent by all map tasks (ms)=11443
                Total vcore-milliseconds taken by all map tasks=11443
                Total megabyte-milliseconds taken by all map tasks=11717632
        Map-Reduce Framework
                Map input records=0
                Map output records=0
                Input split bytes=87
                Spilled Records=0
                Failed Shuffles=0
                Merged Map outputs=0
                GC time elapsed (ms)=538
                CPU time spent (ms)=770
                Physical memory (bytes) snapshot=185745408
                Virtual memory (bytes) snapshot=1606356992
                Total committed heap usage (bytes)=173539328
        File Input Format Counters
                Bytes Read=0
        File Output Format Counters
                Bytes Written=0
26/04/27 02:50:33 INFO mapreduce.ImportJobBase: Transferred 0 bytes in 31.0328 seconds (0 bytes/sec)
26/04/27 02:50:33 INFO mapreduce.ImportJobBase: Retrieved 0 records.
[cloudera@quickstart ~]$ hdfs dfs -cat /user/cloudera/lpu_all/students/part-m-00000
1,Aarav Sharma,85,CSE
2,Priya Singh,78,ECE
3,Rohit Verma,91,CSE
4,Neha Gupta,74,IT
5,Karan Patel,88,ME
6,Anjali Yadav,69,CSE
[cloudera@quickstart ~]$ hdfs dfs -cat /user/cloudera/lpu_all
cat: `/user/cloudera/lpu_all': Is a directory
[cloudera@quickstart ~]$
```

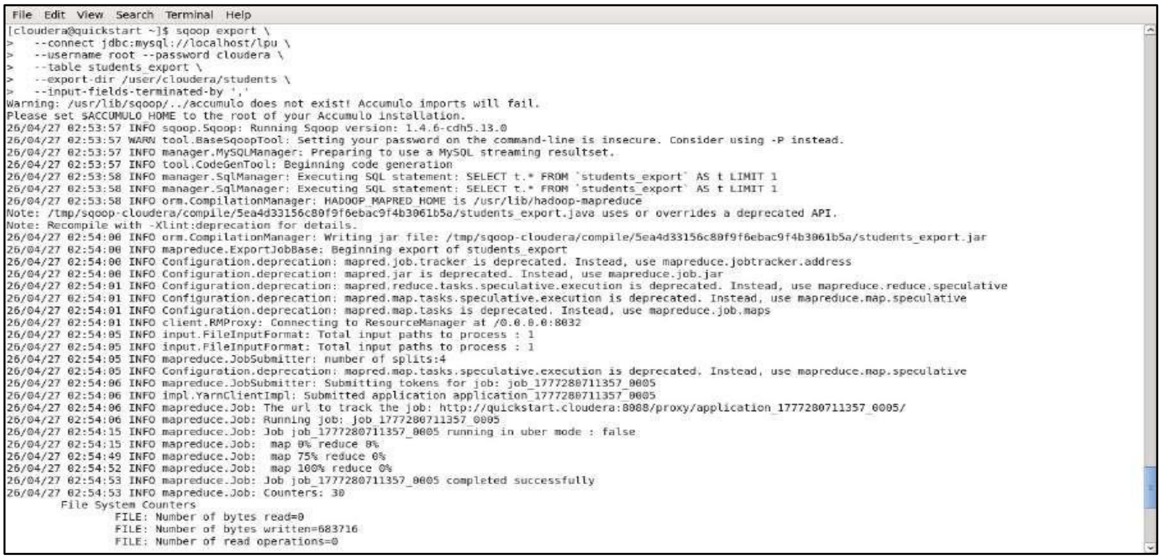
Screenshot 7: Import done. cat on the file shows all 6 students in CSV format inside /user/cloudera/lpu_all/students/.

Q3: Export from HDFS Back to MySQL

Command used:

sqoop export with --table students_export, --export-dir /user/cloudera/students, --input-fields-terminated-by ","

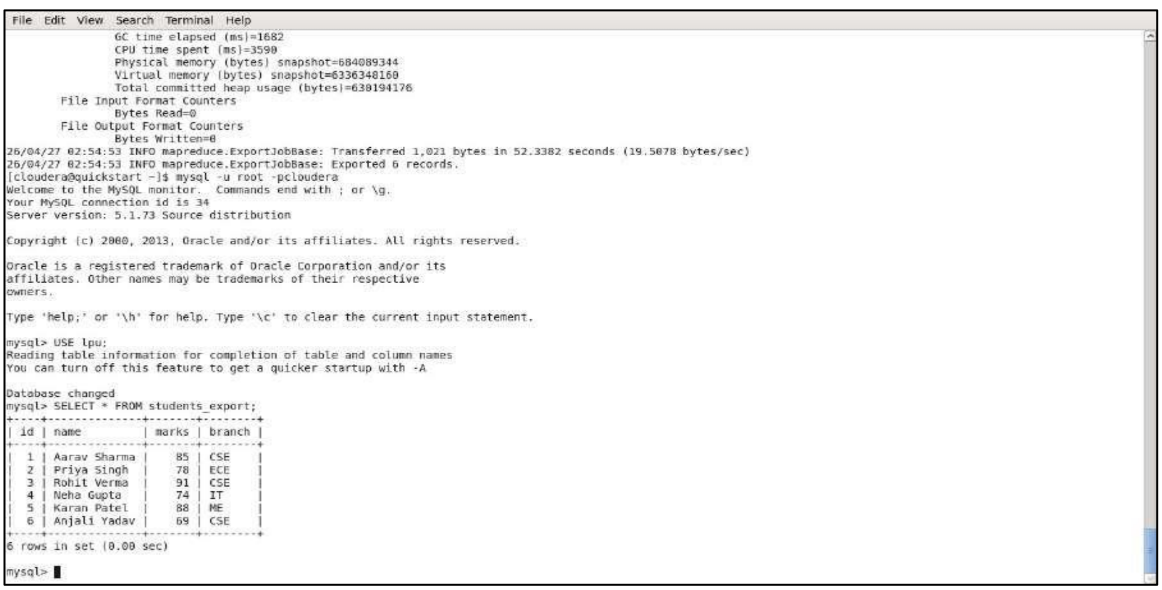
This reads the data from HDFS and pushes it into the MySQL students_export table. The delimiter tells Sqoop that fields in the HDFS file are separated by commas.



```
File  Edit  View  Search  Terminal  Help
[cloudera@quickstart ~]$ sqoop export \
>   --connect jdbc:mysql://localhost/lpu \
>   --username root --password cloudera \
>   --table students_export \
>   --export-dir /user/cloudera/students \
>   --input-fields-terminated-by ','
Warning: /usr/lib/sqoop/../accumulo does not exist! Accumulo imports will fail.
Please set $ACCUMULO_HOME to the root of your Accumulo installation.
26/04/27 02:53:57 INFO sqoop.Sqoop: Running Sqoop version: 1.4.6-cdh5.13.0
26/04/27 02:53:57 WARN tool.BaseSqoopTool: Setting your password on the command-line is insecure. Consider using -P instead.
26/04/27 02:53:57 INFO manager.MySQLManager: Preparing to use a MySQL streaming resultset.
26/04/27 02:53:57 INFO tool.CodeGenTool: Beginning code generation
26/04/27 02:53:58 INFO manager.SqlManager: Executing SQL statement: SELECT t.* FROM `students_export` AS t LIMIT 1
26/04/27 02:53:58 INFO manager.SqlManager: Executing SQL statement: SELECT t.* FROM `students_export` AS t LIMIT 1
26/04/27 02:53:58 INFO orm.CompilationManager: HADOOP_MAPRED_HOME is /usr/lib/hadoop-mapreduce
Note: /tmp/sqoop-cloudera/compile/5ea4d33156c80f9f6ebac9f4b3061b5a/students_export.java uses or overrides a deprecated API.
Note: Recompile with -Xlint:deprecation for details.
26/04/27 02:54:00 INFO orm.CompilationManager: Writing jar file: /tmp/sqoop-cloudera/compile/5ea4d33156c80f9f6ebac9f4b3061b5a/students_export.jar
26/04/27 02:54:00 INFO mapreduce.ExportJobBase: Beginning export of students_export
26/04/27 02:54:00 INFO Configuration.deprecation: mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address
26/04/27 02:54:00 INFO Configuration.deprecation: mapred.jar is deprecated. Instead, use mapreduce.job.jar
26/04/27 02:54:01 INFO Configuration.deprecation: mapred.reduce.tasks.speculative.execution is deprecated. Instead, use mapreduce.reduce.speculative
26/04/27 02:54:01 INFO Configuration.deprecation: mapred.map.tasks.speculative.execution is deprecated. Instead, use mapreduce.map.speculative
26/04/27 02:54:01 INFO Configuration.deprecation: mapred.map.tasks is deprecated. Instead, use mapreduce.job.maps
26/04/27 02:54:01 INFO client.RMProxy: Connecting to ResourceManager at /0.0.0.0:8032
26/04/27 02:54:05 INFO input.FileInputFormat: Total input paths to process : 1
26/04/27 02:54:05 INFO input.FileInputFormat: Total input paths to process : 1
26/04/27 02:54:05 INFO mapreduce.JobSubmitter: number of splits:4
26/04/27 02:54:05 INFO Configuration.deprecation: mapred.map.tasks.speculative.execution is deprecated. Instead, use mapreduce.map.speculative
26/04/27 02:54:06 INFO mapreduce.JobSubmitter: Submitting tokens for job: job_1777280711357_0005
26/04/27 02:54:06 INFO impl.YarnClientImpl: Submitted application application_1777280711357_0005
26/04/27 02:54:06 INFO mapreduce.Job: The url to track the job: http://quickstart.cloudera:8088/proxy/application_1777280711357_0005/
26/04/27 02:54:06 INFO mapreduce.Job: Running job: job_1777280711357_0005
26/04/27 02:54:15 INFO mapreduce.Job: Job job_1777280711357_0005 running in uber mode : false
26/04/27 02:54:15 INFO mapreduce.Job:  map 0% reduce 0%
26/04/27 02:54:49 INFO mapreduce.Job:  map 75% reduce 0%
26/04/27 02:54:52 INFO mapreduce.Job:  map 100% reduce 0%
26/04/27 02:54:53 INFO mapreduce.Job: Job job_1777280711357_0005 completed successfully
26/04/27 02:54:53 INFO mapreduce.Job: Counters: 30
        File System Counters
                FILE: Number of bytes read=0
                FILE: Number of bytes written=683716
                FILE: Number of read operations=0
```

Screenshot 8: Sqoop export job running. 4 splits used, map goes from 0% to 75% to 100%.

- Export finished — 6 records exported. We logged into MySQL and did SELECT * FROM students_export to verify the data came through correctly.



```
File  Edit  View  Search  Terminal  Help
                GC time elapsed (ms)=1682
                CPU time spent (ms)=3590
                Physical memory (bytes) snapshot=684089344
                Virtual memory (bytes) snapshot=6336348160
                Total committed heap usage (bytes)=630194176
        File Input Format Counters
                Bytes Read=0
        File Output Format Counters
                Bytes Written=0
26/04/27 02:54:53 INFO mapreduce.ExportJobBase: Transferred 1,021 bytes in 52.3382 seconds (19.5078 bytes/sec)
26/04/27 02:54:53 INFO mapreduce.ExportJobBase: Exported 6 records.
[cloudera@quickstart ~]$ mysql -u root -pcloudera
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 34
Server version: 5.1.73 Source distribution

Copyright (c) 2000, 2013, Oracle and/or its affiliates. All rights reserved.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE lpu;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SELECT * FROM students_export;
+----+--------------+-------+--------+
| id | name         | marks | branch |
+----+--------------+-------+--------+
|  1 | Aarav Sharma |    85 | CSE    |
|  2 | Priya Singh  |    78 | ECE    |
|  3 | Rohit Verma  |    91 | CSE    |
|  4 | Neha Gupta   |    74 | IT     |
|  5 | Karan Patel  |    88 | ME     |
|  6 | Anjali Yadav |    69 | CSE    |
+----+--------------+-------+--------+
6 rows in set (0.00 sec)

mysql>
```

Screenshot 9: Export done. MySQL shows all 6 students in students_export — exact same data as the original students table.

TASK 2: IMPLEMENTATION OF ANALYTICS COMMANDS USING PIG

Project Definition

The objective of this task is to perform data processing and analytical operations on a dataset using Apache Pig. The task focuses on applying Pig Latin commands to manipulate and analyze data stored in Hadoop, thereby demonstrating data transformation and analysis in a distributed computing environment.

Outcomes / Learning

This task helped in understanding how large datasets stored in HDFS can be processed using Pig Latin. I gained practical knowledge of performing data transformations such as filtering, grouping, sorting, and projection. Additionally, I developed an understanding of how Pig internally executes operations using MapReduce, making it efficient for big data processing.

Required Tools

- **Tool Used:** Apache Pig
- **Technologies Involved:**
    - HDFS (for data storage)
    - Pig Latin (data processing language)
    - MapReduce (execution engine)

Working

Step 1: Data Preparation and Upload to HDFS

A dataset file named students.txt was created locally with comma-separated values containing attributes such as id, name, marks, and branch. This file was uploaded into HDFS using the appropriate command. The Pig shell was then launched to begin processing.

Step 2: Load Data into a Bag

The dataset was loaded into Pig using the LOAD command with PigStorage(','). A schema was defined to assign proper data types to each column. The loaded data was stored in a relation (bag), enabling further operations.

Step 3: FOREACH Command (Projection)

The FOREACH command was used to extract specific columns from the dataset. This operation selected only relevant attributes (id, name, marks), demonstrating data projection similar to SQL queries.

Step 4: GROUP BY Command (Aggregation)

The dataset was grouped based on the branch attribute using the GROUP command. This operation organized data into groups, allowing analysis of records belonging to the same category.

Step 5: ORDER BY Command (Sorting)

The ORDER command was used to sort the dataset based on marks in descending order. This operation helped in identifying top-performing records and demonstrated sorting in distributed systems.

Step 6: FILTER Command (Conditional Selection)

The FILTER command was applied to extract records where marks were greater than a specified threshold. This demonstrated conditional data selection and elimination of irrelevant records.

Output Explanation

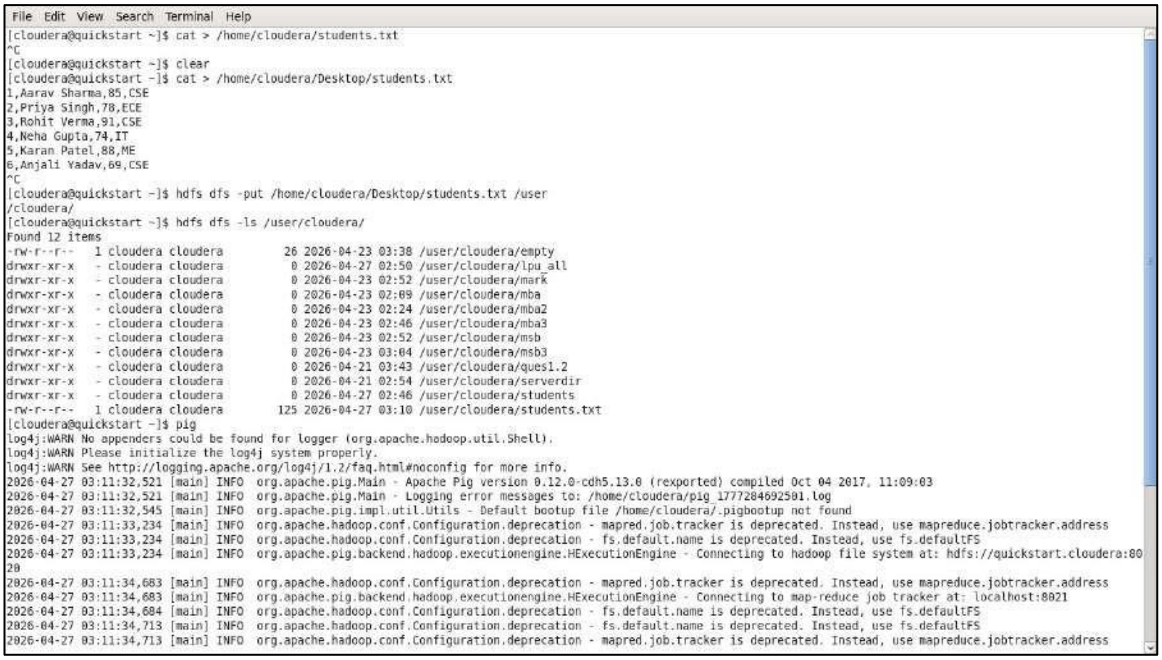
- The dataset was successfully loaded into Pig and displayed using the DUMP command.
- Projection operation correctly reduced the dataset to selected columns.
- Grouping operation categorized records based on branch, showing distribution across categories.
- Sorting operation arranged records in descending order of marks.
- Filtering operation extracted only relevant records based on conditions.
- All operations were executed successfully using Pig's data flow mechanism backed by MapReduce.

Learning Outcome (Task 2)

I learned how to perform efficient data transformation and analysis using Apache Pig. This task enhanced my understanding of data flow processing, schema definition, and execution of analytical operations on large datasets stored in HDFS.

Setup: Create and Upload students.txt to HDFS

We created students.txt on the Desktop using cat command and typed in 6 records (one per line, comma separated). Then uploaded it to HDFS at /user/cloudera/students.txt using hdfs dfs -put. After that we launched the pig shell.



```
File  Edit  View  Search  Terminal  Help
[cloudera@quickstart ~]$ cat > /home/cloudera/students.txt
^C
[cloudera@quickstart ~]$ clear
[cloudera@quickstart ~]$ cat > /home/cloudera/Desktop/students.txt
1,Aarav Sharma,85,CSE
2,Priya Singh,78,ECE
3,Rohit Verma,91,CSE
4,Neha Gupta,74,IT
5,Karan Patel,88,ME
6,Anjali Yadav,69,CSE
^C
[cloudera@quickstart ~]$ hdfs dfs -put /home/cloudera/Desktop/students.txt /user
/cloudera/
[cloudera@quickstart ~]$ hdfs dfs -ls /user/cloudera/
Found 12 items
-rw-r--r--   1 cloudera cloudera         26 2026-04-23 03:38 /user/cloudera/empty
drwxr-xr-x   - cloudera cloudera          0 2026-04-27 02:50 /user/cloudera/lpu_all
drwxr-xr-x   - cloudera cloudera          0 2026-04-23 02:52 /user/cloudera/mark
drwxr-xr-x   - cloudera cloudera          0 2026-04-23 02:09 /user/cloudera/mba
drwxr-xr-x   - cloudera cloudera          0 2026-04-23 02:24 /user/cloudera/mba2
drwxr-xr-x   - cloudera cloudera          0 2026-04-23 02:46 /user/cloudera/mba3
drwxr-xr-x   - cloudera cloudera          0 2026-04-23 02:52 /user/cloudera/msb
drwxr-xr-x   - cloudera cloudera          0 2026-04-23 03:04 /user/cloudera/msb3
drwxr-xr-x   - cloudera cloudera          0 2026-04-21 03:43 /user/cloudera/ques1.2
drwxr-xr-x   - cloudera cloudera          0 2026-04-21 02:54 /user/cloudera/serverdir
drwxr-xr-x   - cloudera cloudera          0 2026-04-27 02:46 /user/cloudera/students
-rw-r--r--   1 cloudera cloudera        125 2026-04-27 03:10 /user/cloudera/students.txt
[cloudera@quickstart ~]$ pig
log4j:WARN No appenders could be found for logger (org.apache.hadoop.util.Shell).
log4j:WARN Please initialize the log4j system properly.
log4j:WARN See http://logging.apache.org/log4j/1.2/faq.html#noconfig for more info.
2026-04-27 03:11:32,521 [main] INFO  org.apache.pig.Main - Apache Pig version 0.12.0-cdh5.13.0 (rexported) compiled Oct 04 2017, 11:09:03
2026-04-27 03:11:32,521 [main] INFO  org.apache.pig.Main - Logging error messages to: /home/cloudera/pig_1777284692501.log
2026-04-27 03:11:32,545 [main] INFO  org.apache.pig.impl.util.Utils - Default bootup file /home/cloudera/.pigbootup not found
2026-04-27 03:11:33,234 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address
2026-04-27 03:11:33,234 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - fs.default.name is deprecated. Instead, use fs.defaultFS
2026-04-27 03:11:33,234 [main] INFO  org.apache.pig.backend.hadoop.executionengine.HExecutionEngine - Connecting to hadoop file system at: hdfs://quickstart.cloudera:80
20
2026-04-27 03:11:34,683 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address
2026-04-27 03:11:34,683 [main] INFO  org.apache.pig.backend.hadoop.executionengine.HExecutionEngine - Connecting to map-reduce job tracker at: localhost:8021
2026-04-27 03:11:34,684 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - fs.default.name is deprecated. Instead, use fs.defaultFS
2026-04-27 03:11:34,713 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - fs.default.name is deprecated. Instead, use fs.defaultFS
2026-04-27 03:11:34,713 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address
```

Screenshot 1: students.txt created, uploaded to HDFS, confirmed with hdfs dfs -ls. Pig shell launched.

Command 1: LOAD — Load Data into a Bag

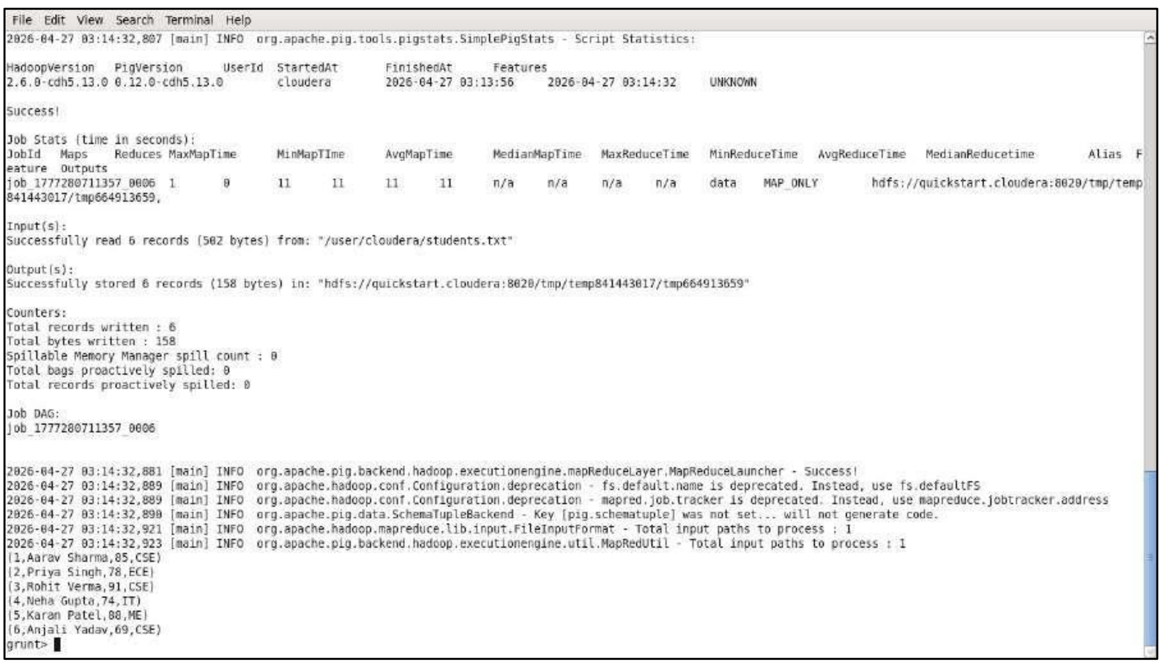
Command:

```
data = LOAD '/user/cloudera/students.txt' USING PigStorage(',') AS (id:int, name:chararray, marks:int, branch:chararray);
```

This reads the CSV file from HDFS and maps each column to a name and type. The result is stored in a relation called data. DUMP data shows all 6 records.

```
File  Edit  View  Search  Terminal  Help
grunt> data = LOAD '/user/cloudera/students.txt' USING PigStorage(',') AS (id:int, name:chararray, marks:int, branch:chararray);
grunt> DUMP data;
2026-04-27 03:13:54,808 [main] INFO  org.apache.pig.tools.pigstats.ScriptState - Pig features used in the script: UNKNOWN
2026-04-27 03:13:54,870 [main] INFO  org.apache.pig.newplan.logical.optimizer.LogicalPlanOptimizer - {RULES_ENABLED=[AddForEach, ColumnMapKeyPrune, DuplicateForEachColu
mnRewrite, GroupByConstParallelSetter, ImplicitSplitInserter, LimitOptimizer, LoadTypeCastInserter, MergeFilter, MergeForEach, NewPartitionFilterOptimizer, PushDownForE
achFlatten, PushUpFilter, SplitFilter, StreamTypeCastInserter], RULES_DISABLED=[FilterLogicExpressionSimplifier, PartitionFilterOptimizer]}
2026-04-27 03:13:55,084 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MRCompiler - File concatenation threshold: 100 optimistic? false
2026-04-27 03:13:55,183 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MultiQueryOptimizer - MR plan size before optimization: 1
2026-04-27 03:13:55,183 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MultiQueryOptimizer - MR plan size after optimization: 1
2026-04-27 03:13:55,858 [main] INFO  org.apache.hadoop.yarn.client.RMProxy - Connecting to ResourceManager at /0.0.0.0:8032
2026-04-27 03:13:56,469 [main] INFO  org.apache.pig.tools.pigstats.ScriptState - Pig script settings are added to the job
2026-04-27 03:13:56,557 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - mapred.job.reduce.markreset.buffer.percent is deprecated. Instead, use mapreduce
.reduce.markreset.buffer.percent
2026-04-27 03:13:56,557 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.JobControlCompiler - mapred.job.reduce.markreset.buffer.percent is not
 set, set to default 0.3
2026-04-27 03:13:56,557 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - mapred.output.compress is deprecated. Instead, use mapreduce.output.fileoutputfo
rmat.compress
2026-04-27 03:13:58,104 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.JobControlCompiler - creating jar file Job748545268951916960.jar
2026-04-27 03:14:00,682 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.JobControlCompiler - jar file Job748545268951916960.jar created
2026-04-27 03:14:00,682 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - mapred.jar is deprecated. Instead, use mapreduce.job.jar
2026-04-27 03:14:00,705 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.JobControlCompiler - Setting up single store job
2026-04-27 03:14:00,711 [main] INFO  org.apache.pig.data.SchemaTupleFrontend - Key [pig.schematuple] is false, will not generate code.
2026-04-27 03:14:00,711 [main] INFO  org.apache.pig.data.SchemaTupleFrontend - Starting process to move generated code to distributed cache
2026-04-27 03:14:00,711 [main] INFO  org.apache.pig.data.SchemaTupleFrontend - Setting key [pig.schematuple.classes] with classes to deserialize []
2026-04-27 03:14:00,737 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - 1 map-reduce job(s) waiting for submission.
2026-04-27 03:14:00,738 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker.http.address is deprecated. Instead, use mapreduce.jobtracker
.http.address
2026-04-27 03:14:00,738 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address
2026-04-27 03:14:00,754 [JobControl] INFO  org.apache.hadoop.yarn.client.RMProxy - Connecting to ResourceManager at /0.0.0.0:8032
2026-04-27 03:14:00,779 [JobControl] INFO  org.apache.hadoop.conf.Configuration.deprecation - fs.default.name is deprecated. Instead, use fs.defaultFS
2026-04-27 03:14:01,237 [JobControl] INFO  org.apache.hadoop.mapreduce.lib.input.FileInputFormat - Total input paths to process : 1
2026-04-27 03:14:01,237 [JobControl] INFO  org.apache.pig.backend.hadoop.executionengine.util.MapRedUtil - Total input paths to process : 1
2026-04-27 03:14:01,264 [JobControl] INFO  org.apache.pig.backend.hadoop.executionengine.util.MapRedUtil - Total input paths (combined) to process : 1
2026-04-27 03:14:01,743 [JobControl] INFO  org.apache.hadoop.mapreduce.JobSubmitter - number of splits:1
2026-04-27 03:14:01,936 [JobControl] INFO  org.apache.hadoop.mapreduce.JobSubmitter - Submitting tokens for job: job_1777280711357_0006
2026-04-27 03:14:02,401 [JobControl] INFO  org.apache.hadoop.yarn.client.api.impl.YarnClientImpl - Submitted application application_1777280711357_0006
2026-04-27 03:14:02,446 [JobControl] INFO  org.apache.hadoop.mapreduce.Job - The url to track the job: http://quickstart.cloudera:8088/proxy/application_1777280711357_0
006/
2026-04-27 03:14:02,447 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - HadoopJobId: job_1777280711357_0006
2026-04-27 03:14:02,447 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - Processing aliases data
2026-04-27 03:14:02,447 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - detailed locations: M: data[1,7],data[-1,-1] C:  R
:
2026-04-27 03:14:02,447 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - More information at: http://localhost:50030/jobdet
```

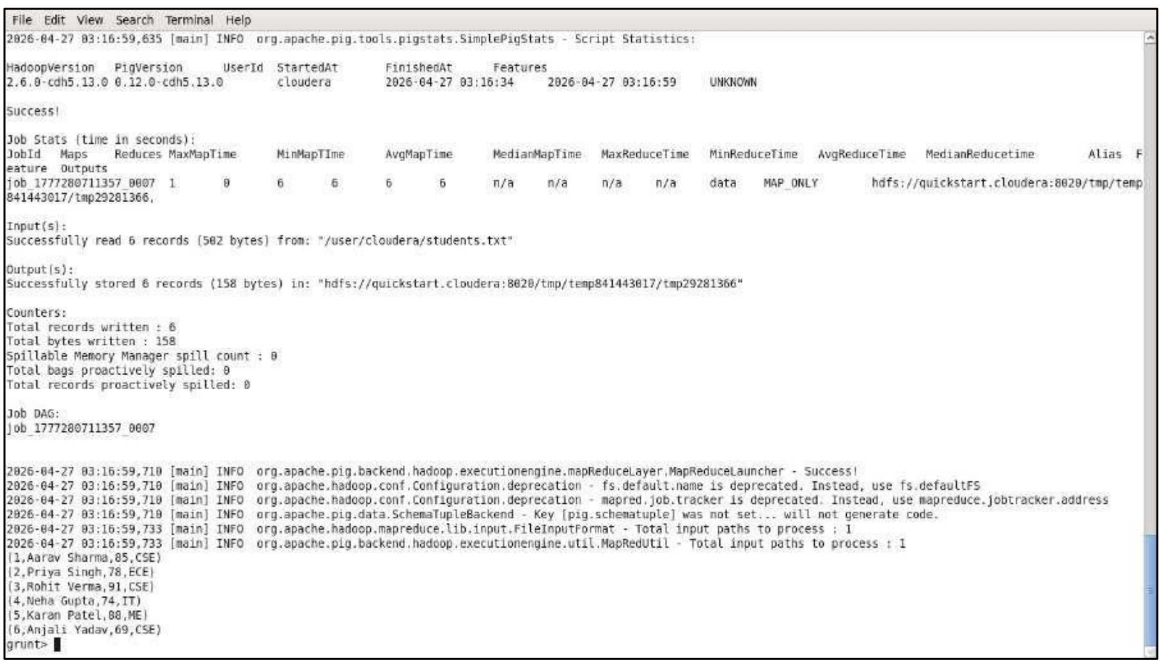
Screenshot 2: LOAD command runs. MapReduce job fires up to process the file.



```
File  Edit  View  Search  Terminal  Help
2026-04-27 03:14:32,807 [main] INFO  org.apache.pig.tools.pigstats.SimplePigStats - Script Statistics:

HadoopVersion   PigVersion      UserId  StartedAt       FinishedAt      Features
2.6.0-cdh5.13.0 0.12.0-cdh5.13.0        cloudera        2026-04-27 03:13:56     2026-04-27 03:14:32     UNKNOWN

Success!

Job Stats (time in seconds):
JobId   Maps    Reduces MaxMapTime      MinMapTIme      AvgMapTime      MedianMapTime   MaxReduceTime   MinReduceTime   AvgReduceTime   MedianReducetime        Alias   F
eature  Outputs
job_1777280711357_0006  1       0       11      11      11      11      n/a     n/a     n/a     n/a     data    MAP_ONLY        hdfs://quickstart.cloudera:8020/tmp/temp
841443017/tmp664913659,

Input(s):
Successfully read 6 records (502 bytes) from: "/user/cloudera/students.txt"

Output(s):
Successfully stored 6 records (158 bytes) in: "hdfs://quickstart.cloudera:8020/tmp/temp841443017/tmp664913659"

Counters:
Total records written : 6
Total bytes written : 158
Spillable Memory Manager spill count : 0
Total bags proactively spilled: 0
Total records proactively spilled: 0

Job DAG:
job_1777280711357_0006


2026-04-27 03:14:32,881 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - Success!
2026-04-27 03:14:32,889 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - fs.default.name is deprecated. Instead, use fs.defaultFS
2026-04-27 03:14:32,889 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address
2026-04-27 03:14:32,890 [main] INFO  org.apache.pig.data.SchemaTupleBackend - Key [pig.schematuple] was not set... will not generate code.
2026-04-27 03:14:32,921 [main] INFO  org.apache.hadoop.mapreduce.lib.input.FileInputFormat - Total input paths to process : 1
2026-04-27 03:14:32,923 [main] INFO  org.apache.pig.backend.hadoop.executionengine.util.MapRedUtil - Total input paths to process : 1
(1,Aarav Sharma,85,CSE)
(2,Priya Singh,78,ECE)
(3,Rohit Verma,91,CSE)
(4,Neha Gupta,74,IT)
(5,Karan Patel,88,ME)
(6,Anjali Yadav,69,CSE)
grunt>
```

Screenshot 3: DUMP result — all 6 student records loaded correctly from the file.

Screenshot 4: LOAD run again in a fresh session before running DUMP.



Screenshot 5: DUMP data confirmed — 6 rows with id, name, marks, branch.

Command 2: FOREACH — Select Specific Columns

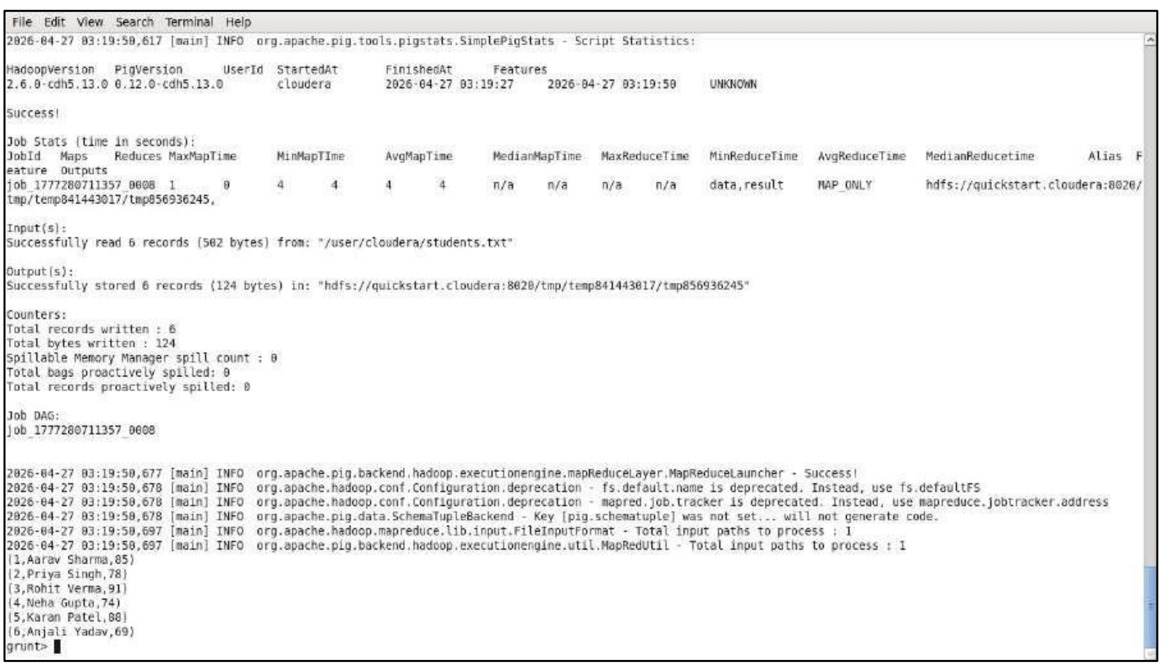
Command:

result = FOREACH data GENERATE id, name, marks;

FOREACH goes through every row in data and picks only id, name, and marks — dropping the branch column. Like a SELECT in SQL.

```
File  Edit  View  Search  Terminal  Help
grunt> result = FOREACH data GENERATE id, name, marks;
grunt> DUMP result;
2026-04-27 03:19:26,993 [main] INFO  org.apache.pig.tools.pigstats.ScriptState - Pig features used in the script: UNKNOWN
2026-04-27 03:19:26,996 [main] INFO  org.apache.pig.newplan.logical.optimizer.LogicalPlanOptimizer - {RULES_ENABLED=[AddForEach, ColumnMapKeyPrune, DuplicateForEachColu
mnRewrite, GroupByConstParallelSetter, ImplicitSplitInserter, LimitOptimizer, LoadTypeCastInserter, MergeFilter, MergeForEach, NewPartitionFilterOptimizer, PushDownForE
achFlatten, PushUpFilter, SplitFilter, StreamTypeCastInserter], RULES_DISABLED=[FilterLogicExpressionSimplifier, PartitionFilterOptimizer]}
2026-04-27 03:19:27,002 [main] INFO  org.apache.pig.newplan.logical.rules.ColumnPruneVisitor - Columns pruned for data: $3
2026-04-27 03:19:27,012 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MRCompiler - File concatenation threshold: 100 optimistic? false
2026-04-27 03:19:27,013 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MultiQueryOptimizer - MR plan size before optimization: 1
2026-04-27 03:19:27,013 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MultiQueryOptimizer - MR plan size after optimization: 1
2026-04-27 03:19:27,038 [main] INFO  org.apache.hadoop.yarn.client.RMProxy - Connecting to ResourceManager at /0.0.0.0:8032
2026-04-27 03:19:27,040 [main] INFO  org.apache.pig.tools.pigstats.ScriptState - Pig script settings are added to the job
2026-04-27 03:19:27,048 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.JobControlCompiler - mapred.job.reduce.markreset.buffer.percent is not
 set, set to default 0.3
2026-04-27 03:19:27,544 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.JobControlCompiler - creating jar file Job2488945110902679903.jar
2026-04-27 03:19:30,128 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.JobControlCompiler - jar file Job2488945110902679903.jar created
2026-04-27 03:19:30,136 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.JobControlCompiler - Setting up single store job
2026-04-27 03:19:30,138 [main] INFO  org.apache.pig.data.SchemaTupleFrontend - Key [pig.schematuple] is false, will not generate code.
2026-04-27 03:19:30,138 [main] INFO  org.apache.pig.data.SchemaTupleFrontend - Starting process to move generated code to distributed cache
2026-04-27 03:19:30,138 [main] INFO  org.apache.pig.data.SchemaTupleFrontend - Setting key [pig.schematuple.classes] with classes to deserialize []
2026-04-27 03:19:30,150 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - 1 map-reduce job(s) waiting for submission.
2026-04-27 03:19:30,150 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address
2026-04-27 03:19:30,154 [JobControl] INFO  org.apache.hadoop.yarn.client.RMProxy - Connecting to ResourceManager at /0.0.0.0:8032
2026-04-27 03:19:30,157 [JobControl] INFO  org.apache.hadoop.conf.Configuration.deprecation - fs.default.name is deprecated. Instead, use fs.defaultFS
2026-04-27 03:19:30,305 [JobControl] INFO  org.apache.hadoop.mapreduce.lib.input.FileInputFormat - Total input paths to process : 1
2026-04-27 03:19:30,305 [JobControl] INFO  org.apache.pig.backend.hadoop.executionengine.util.MapRedUtil - Total input paths to process : 1
2026-04-27 03:19:30,306 [JobControl] INFO  org.apache.pig.backend.hadoop.executionengine.util.MapRedUtil - Total input paths (combined) to process : 1
2026-04-27 03:19:30,347 [JobControl] INFO  org.apache.hadoop.mapreduce.JobSubmitter - number of splits:1
2026-04-27 03:19:30,388 [JobControl] INFO  org.apache.hadoop.mapreduce.JobSubmitter - Submitting tokens for job: job_1777280711357_0008
2026-04-27 03:19:30,459 [JobControl] INFO  org.apache.hadoop.yarn.client.api.impl.YarnClientImpl - Submitted application application_1777280711357_0008
2026-04-27 03:19:30,465 [JobControl] INFO  org.apache.hadoop.mapreduce.Job - The url to track the job: http://quickstart.cloudera:8088/proxy/application_1777280711357_0
008/
2026-04-27 03:19:30,652 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - HadoopJobId: job_1777280711357_0008
2026-04-27 03:19:30,652 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - Processing aliases data,result
2026-04-27 03:19:30,652 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - detailed locations: M: data[2,7],result[-1,-1] C:
 R:
2026-04-27 03:19:30,652 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - More information at: http://localhost:50030/jobdet
ails.jsp?jobid=job_1777280711357_0008
2026-04-27 03:19:30,684 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - 0% complete
2026-04-27 03:19:44,705 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - 50% complete
2026-04-27 03:19:50,616 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - 100% complete
2026-04-27 03:19:50,617 [main] INFO  org.apache.pig.tools.pigstats.SimplePigStats - Script Statistics:
```

Screenshot 6: FOREACH command runs with MapReduce job.



```
File  Edit  View  Search  Terminal  Help
2026-04-27 03:19:50,617 [main] INFO  org.apache.pig.tools.pigstats.SimplePigStats - Script Statistics:

HadoopVersion   PigVersion      UserId  StartedAt       FinishedAt      Features
2.6.0-cdh5.13.0 0.12.0-cdh5.13.0        cloudera        2026-04-27 03:19:27     2026-04-27 03:19:50     UNKNOWN

Success!

Job Stats (time in seconds):
JobId   Maps    Reduces MaxMapTime      MinMapTIme      AvgMapTime      MedianMapTime   MaxReduceTime   MinReduceTime   AvgReduceTime   MedianReducetime        Alias   F
eature  Outputs
job_1777280711357_0008  1       0       4       4       4       4       n/a     n/a     n/a     n/a     data,result     MAP_ONLY        hdfs://quickstart.cloudera:8020/
tmp/temp841443017/tmp856936245,

Input(s):
Successfully read 6 records (502 bytes) from: "/user/cloudera/students.txt"

Output(s):
Successfully stored 6 records (124 bytes) in: "hdfs://quickstart.cloudera:8020/tmp/temp841443017/tmp856936245"

Counters:
Total records written : 6
Total bytes written : 124
Spillable Memory Manager spill count : 0
Total bags proactively spilled: 0
Total records proactively spilled: 0

Job DAG:
job_1777280711357_0008


2026-04-27 03:19:50,677 [main] INFO  org.apache.pig.backend.hadoop.executionengine.mapReduceLayer.MapReduceLauncher - Success!
2026-04-27 03:19:50,678 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - fs.default.name is deprecated. Instead, use fs.defaultFS
2026-04-27 03:19:50,678 [main] INFO  org.apache.hadoop.conf.Configuration.deprecation - mapred.job.tracker is deprecated. Instead, use mapreduce.jobtracker.address
2026-04-27 03:19:50,678 [main] INFO  org.apache.pig.data.SchemaTupleBackend - Key [pig.schematuple] was not set... will not generate code.
2026-04-27 03:19:50,697 [main] INFO  org.apache.hadoop.mapreduce.lib.input.FileInputFormat - Total input paths to process : 1
2026-04-27 03:19:50,697 [main] INFO  org.apache.pig.backend.hadoop.executionengine.util.MapRedUtil - Total input paths to process : 1
(1,Aarav Sharma,85)
(2,Priya Singh,78)
(3,Rohit Verma,91)
(4,Neha Gupta,74)
(5,Karan Patel,88)
(6,Anjali Yadav,69)
grunt>
```

Screenshot 7: DUMP result shows 6 rows with only id, name, marks — branch column removed.

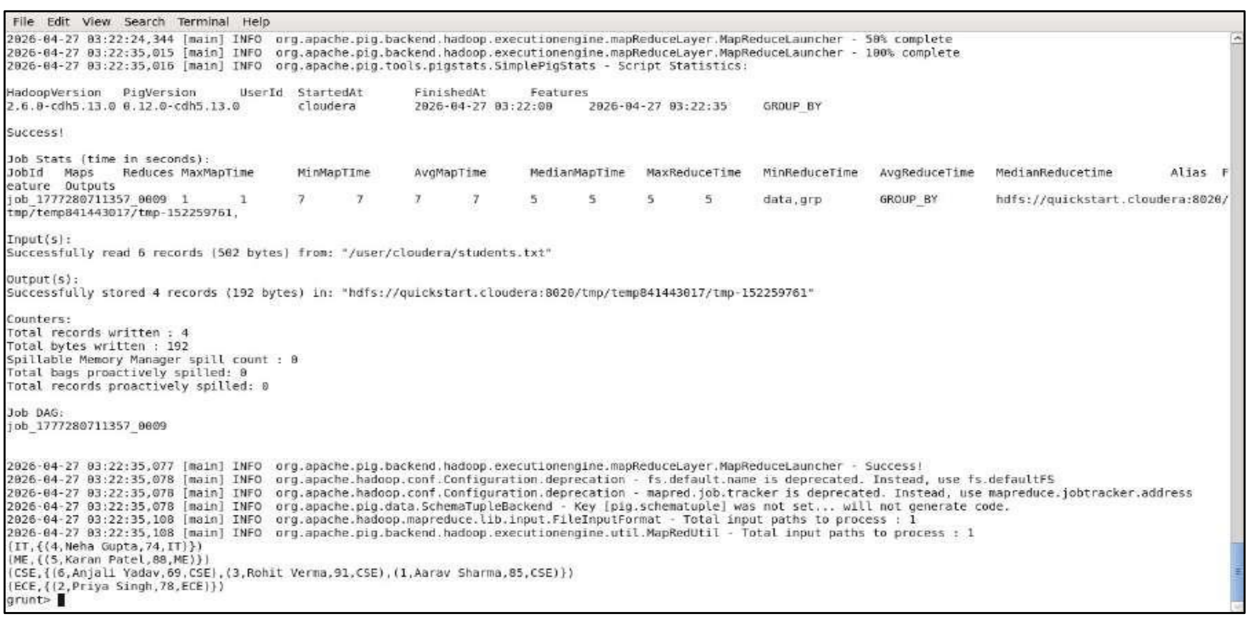
Command 3: GROUP BY — Group Students by Branch

Command:

grp = GROUP data BY branch;

This groups all students by their branch. Students in the same branch get collected into a bag together. CSE gets 3 students, ECE gets 1, IT gets 1, ME gets 1.

Screenshot 8: GROUP BY command runs. Pig fires a MapReduce job with a reduce phase this time.



Screenshot 9: DUMP grp — 4 groups: IT, ME, CSE (3 students), ECE. Each group shows its bag of records.

Command 4: ORDER BY — Sort by Marks Descending

Command:

ordered = ORDER data BY marks DESC;

This sorts all 6 students from highest to lowest marks. Rohit (91) comes first, Anjali (69) comes last.

Screenshot 10: ORDER BY command runs. Multiple MapReduce jobs fire to handle the sort.

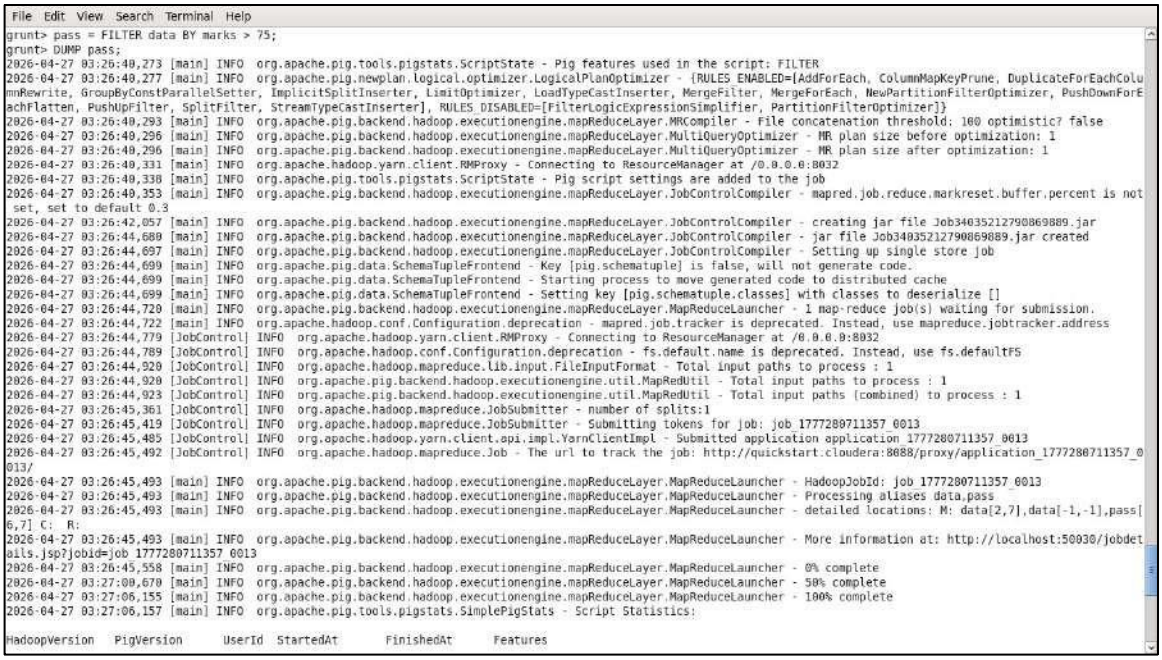
Screenshot 11: DUMP ordered — students sorted by marks descending: 91, 88, 85, 78, 74, 69.

Command 5: FILTER — Keep Only Marks > 75

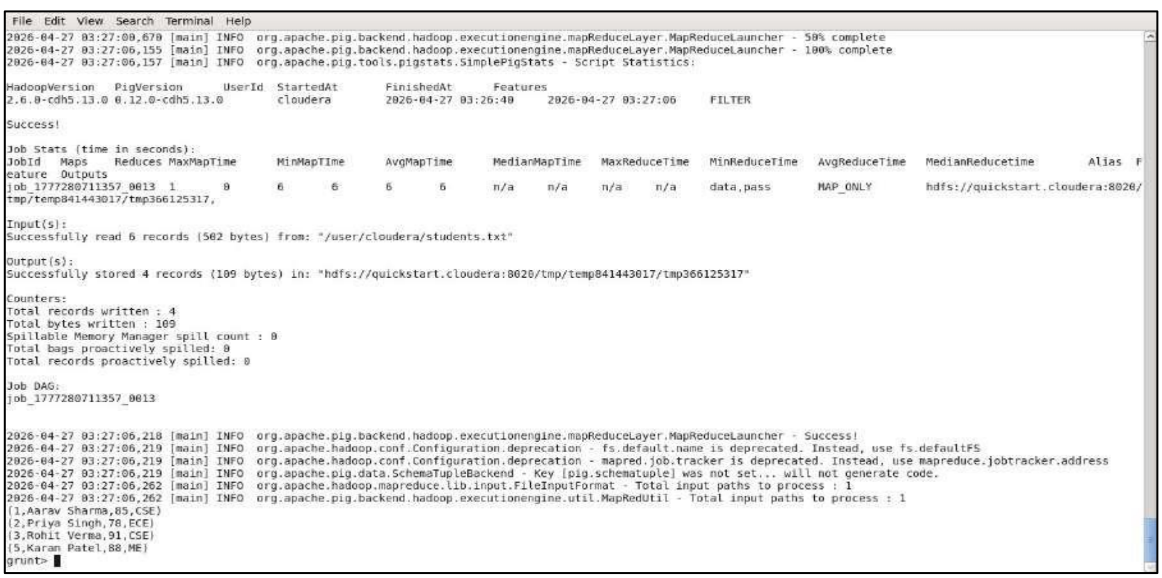
Command:

pass = FILTER data BY marks > 75;

This filters out students who scored 75 or below. Only 4 students pass: Aarav (85), Priya (78), Rohit (91), Karan
(88). Neha (74) and Anjali (69) are excluded.



Screenshot 12: FILTER command runs.



Screenshot 13: DUMP pass — 4 records where marks > 75. Neha and Anjali are correctly filtered out.

TASK 3: IMPLEMENTATION OF ANALYTICS OPERATIONS USING HIVE

Project Definition

The objective of this task is to implement data warehousing operations using Apache Hive. This includes creating internal and external tables, loading data from the local system into Hive, and performing static and dynamic partitioning. The task demonstrates how Hive enables structured data storage and efficient querying over large datasets in Hadoop.

Outcomes / Learning

This task provided a clear understanding of how Hive transforms raw data stored in HDFS into a structured, queryable format using HiveQL. I gained practical exposure to table creation, data loading techniques, and partitioning strategies. Additionally, I understood how partitioning improves query performance by minimizing data scanning.

Required Tools

- **Tool Used:** Apache Hive
- **Technologies Involved:**
    - HiveQL (SQL-like query language)
    - HDFS (data storage layer)
    - Hadoop ecosystem

Working

Step 1: Create Internal Table and Load Data

An internal (managed) table named students_internal was created in Hive with columns such as id, name, marks, and branch. Data was loaded from the local system into the table using the LOAD DATA command. In this case, Hive manages both the table metadata and the actual data. If the table is deleted, the data is also removed from storage.

Step 2: Create External Table and Load Data

An external table named students_external was created with a specified HDFS location. Unlike internal tables, Hive only maintains metadata for external tables, while the actual data remains stored in HDFS.
This approach is useful when the same dataset needs to be accessed by multiple tools or when data persistence is required even after table deletion.

Step 3: Static Partitioning

A partitioned table named students_static was created, where partitioning was based on the branch attribute. Data was inserted into a specific partition manually (e.g., branch = 'CSE').
Static partitioning requires explicit specification of partition values during insertion, allowing controlled and precise data organization.

Step 4: Dynamic Partitioning

Dynamic partitioning was implemented by enabling Hive settings and inserting data without explicitly specifying partition values. Hive automatically created partitions based on the branch column values present in the dataset.
This method simplifies data loading when multiple partition values exist and enhances scalability.

Output Explanation

- The internal table successfully stored and displayed all records loaded from the local file.
- The external table correctly referenced data stored in HDFS without duplicating it.
- Static partitioning created a specific partition and stored only relevant records.
- Dynamic partitioning automatically generated multiple partitions based on data values.
- Query results confirmed that all records were correctly stored and retrieved across different table types.

Learning Outcome (Task 3)

I learned how to use Hive for structured data management in a big data environment. This task strengthened my understanding of table types, data loading techniques, and partitioning strategies, which are essential for optimizing query performance and managing large datasets efficiently.

Internal Table — students_internal

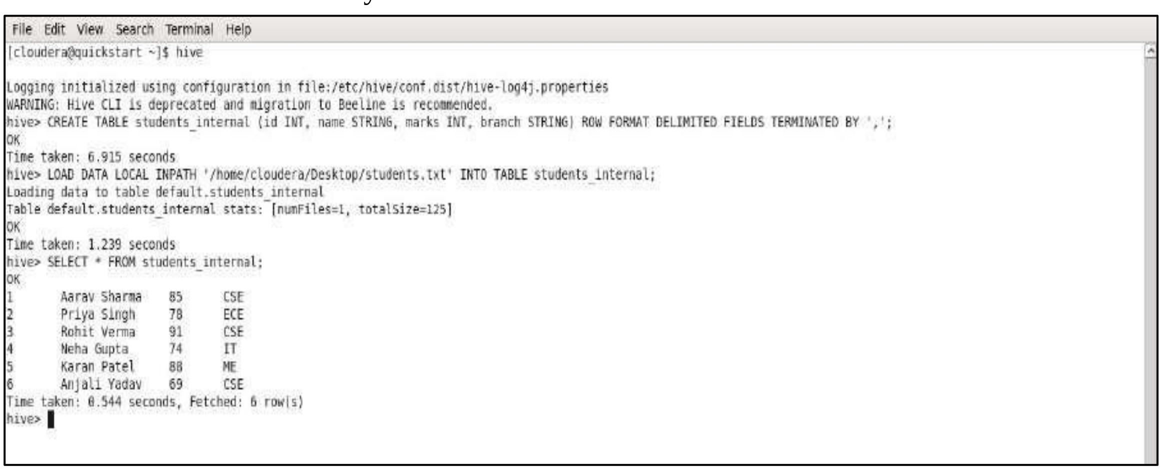
Command:

CREATE TABLE students_internal (id INT, name STRING, marks INT, branch STRING) ROW FORMAT DELIMITED FIELDS TERMINATED BY ',';

Then:

LOAD DATA LOCAL INPATH '/home/cloudera/Desktop/students.txt' INTO TABLE students_internal;

An internal (managed) table means Hive owns the data. If you drop the table, the data is deleted too.
We loaded the local file directly into it.



```
File  Edit  View  Search  Terminal  Help
[cloudera@quickstart ~]$ hive

Logging initialized using configuration in file:/etc/hive/conf.dist/hive-log4j.properties
WARNING: Hive CLI is deprecated and migration to Beeline is recommended.
hive> CREATE TABLE students_internal (id INT, name STRING, marks INT, branch STRING) ROW FORMAT DELIMITED FIELDS TERMINATED BY ',';
OK
Time taken: 6.915 seconds
hive> LOAD DATA LOCAL INPATH '/home/cloudera/Desktop/students.txt' INTO TABLE students_internal;
Loading data to table default.students_internal
Table default.students_internal stats: [numFiles=1, totalSize=125]
OK
Time taken: 1.239 seconds
hive> SELECT * FROM students_internal;
OK
1       Aarav Sharma    85      CSE
2       Priya Singh     78      ECE
3       Rohit Verma     91      CSE
4       Neha Gupta      74      IT
5       Karan Patel     88      ME
6       Anjali Yadav    69      CSE
Time taken: 0.544 seconds, Fetched: 6 row(s)
hive>
```

*Screenshot 1: Internal table created. File loaded. SELECT * shows all 6 students.*

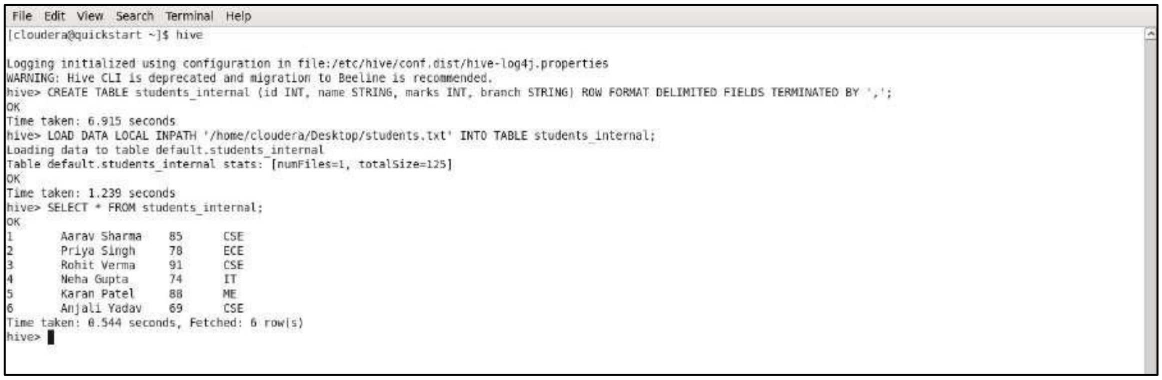
External Table — students_external

Command:

CREATE EXTERNAL TABLE students_external (id INT, name STRING, marks INT, branch STRING) ROW FORMAT DELIMITED FIELDS TERMINATED BY ','

LOCATION '/user/cloudera/external_students';

An external table means Hive just points to data in HDFS. If you drop the table, the actual data in HDFS stays safe. Good when multiple tools need to access the same data.



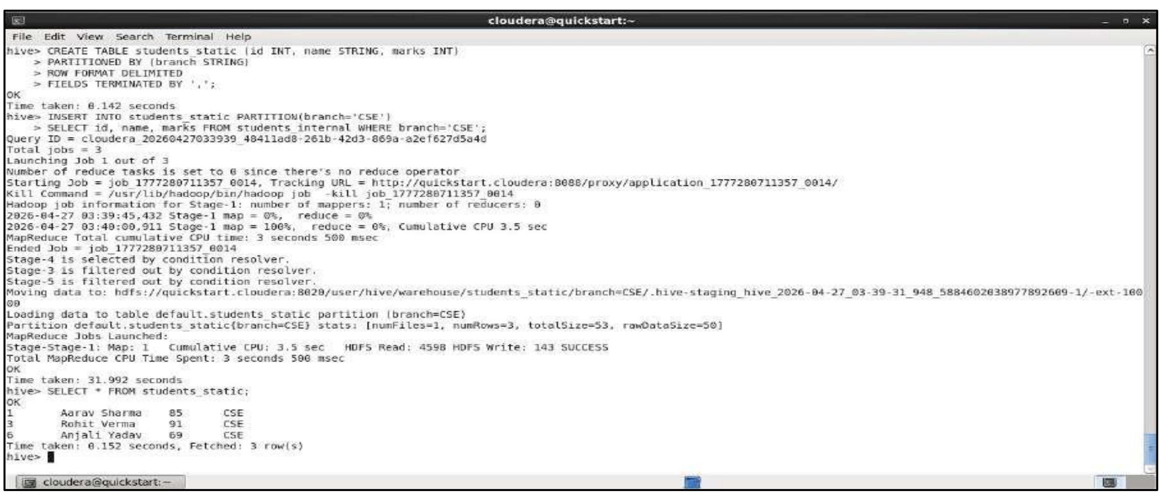
*Screenshot 2: External table created with HDFS location. Data loaded. SELECT * confirms all 6 rows.*

Static Partition — students_static

Commands:

CREATE TABLE students_static (id INT, name STRING, marks INT) PARTITIONED BY (branch STRING). Then INSERT INTO students_static PARTITION(branch='CSE') SELECT id, name, marks FROM students_internal WHERE branch='CSE';

Static partitioning means you manually tell Hive which partition to write into. We pushed only the CSE students (Aarav, Rohit, Anjali) into the CSE partition. The SELECT result shows those 3 rows.



*Screenshot 3: Static partition table created. CSE partition inserted. SELECT * shows 3 CSE students — Aarav, Rohit, Anjali.*

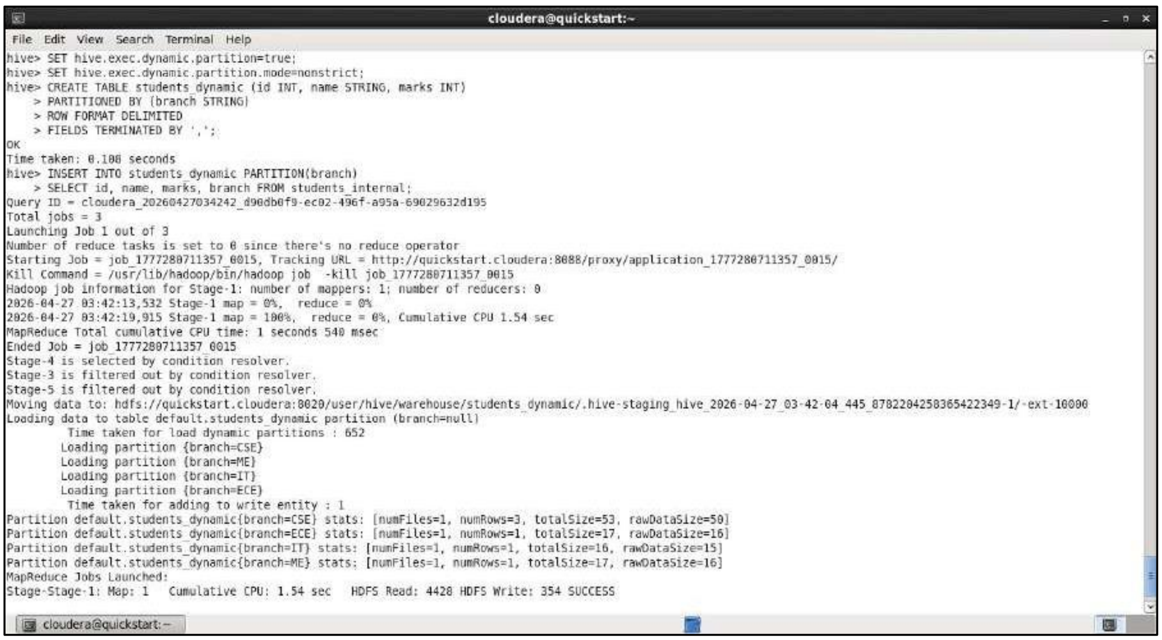
Dynamic Partition — students_dynamic

Commands:

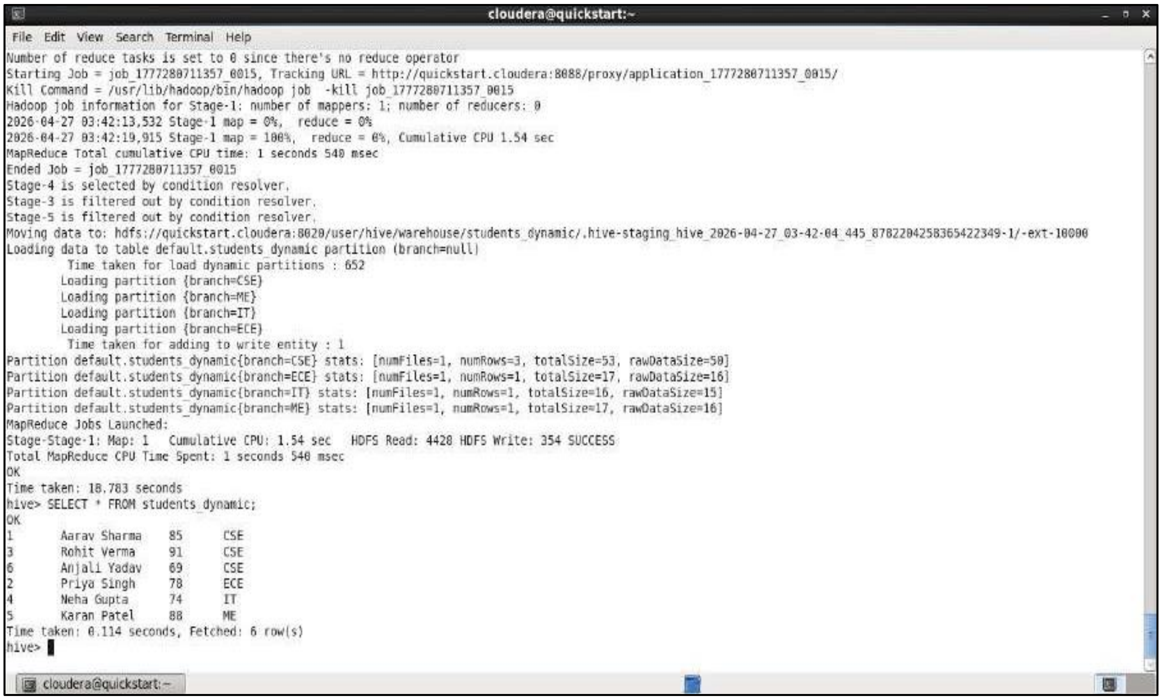
SET hive.exec.dynamic.partition=true; SET hive.exec.dynamic.partition.mode=nonstrict; Then CREATE TABLE students_dynamic (same structure) PARTITIONED BY (branch STRING).

Then INSERT INTO students_dynamic PARTITION(branch) SELECT id, name, marks, branch FROM students_internal;

Dynamic partitioning figures out the partition value automatically from the data. Hive created 4 partitions (CSE, ECE, IT, ME) all at once from the single INSERT. Much easier than static when you have many partition values.



Screenshot 4: Dynamic partitions loading — CSE, ME, IT, ECE all created automatically. Each partition shows correct row counts.



*Screenshot 5: SELECT * FROM students_dynamic — all 6 students returned, sorted by branch partition.*

Final Conclusion

This assignment provided comprehensive practical exposure to core components of the Hadoop ecosystem, including data ingestion, processing, and warehousing. Through the implementation of Sqoop, Pig, and Hive, I was able to understand the complete lifecycle of big data handling, starting from transferring structured data from relational databases into HDFS, performing analytical transformations, and finally organizing data for efficient querying.

The Sqoop operations helped in establishing a strong foundation in data integration by enabling seamless import and export between MySQL and Hadoop. The Pig tasks enhanced my understanding of data flow processing and demonstrated how large datasets can be efficiently transformed using high-level scripting. Furthermore, the Hive operations introduced the concept of structured data management, where internal and external tables, along with partitioning techniques, significantly improved data organization and query performance.

Overall, this assignment strengthened my practical knowledge of big data tools and their real-world applications. It also improved my ability to handle large datasets, optimize data processing workflows, and apply analytical techniques in a distributed environment. These skills are essential for building scalable data solutions and are highly relevant in modern data-driven industries.